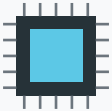
Computer Architecture • Chapter 5

Modern Processor Architectures



multi-core, parallelism, Amdahl's law, cache memory, memory hierarchy

Teacher: Dr. Benabderrezak Youcef

Course: UEF31, Architecture des ordinateurs

Goals of this chapter

At the end, you can:

- explain why we moved to multi-core processors;
- compute a speed-up with Amdahl's law;
- explain cache memory, locality and the average access time.

All numbers measured on an Intel Core i7-8665U (4 cores, 8 threads).

Fifty years of progress

Processor	Year	Transistors	Frequency	Cores
Intel 4004	1971	2 300	0.74 MHz	1
Intel 8086	1978	29 000	5–10 MHz	1
Pentium 4	2000	42 million	1.3–3.8 GHz	1
Core i7-8665U	2019	about 2 billion	1.9–4.8 GHz	4
2024 processors	2024	tens of billions	about 5 GHz	8 to 100+

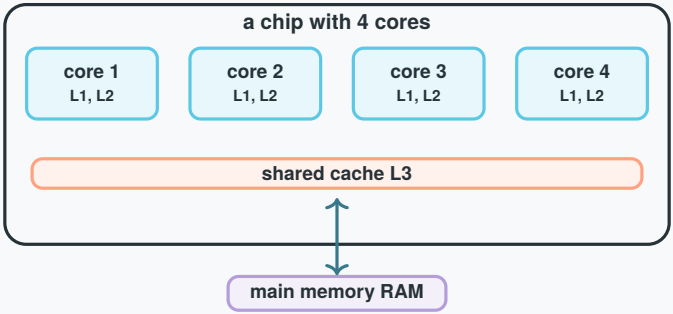
The frequency wall

- Until 2004, the frequency went up every year.
- Then it stopped near 4–5 GHz: too much **heat**.
- So makers added **more cores**.

Analogy

One runner has a top speed: we add more runners.

Cores and threads



A **core** is a full processor. A **thread** is a series of instructions that runs on its own.

Hyper-Threading

- One core runs **two threads**: when one waits, the other works.
- Our processor: 4 cores, seen as 8 logical processors.
- If both threads need the same unit, we gain almost nothing.

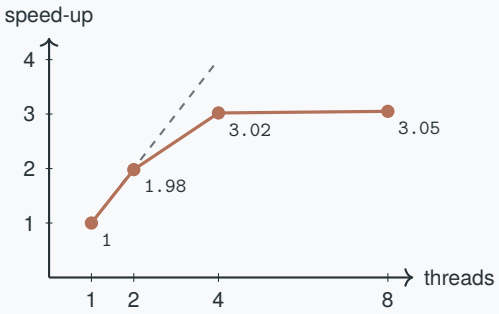
A test: the same work on 1, 2, 4 and 8 threads

```
18 static void *worker(void *arg)      /* thread id computes its part */
19 {
20     long id = (long) arg;
21     double s = 0;
22     for (long i = id; i < WORK; i += k)
23         s += sqrt((double) i);
24     part[id] = s;
25     return NULL;
26 }
```

```
1 thread(s): 0.224 s  speedup 1.00
2 thread(s): 0.113 s  speedup 1.98
4 thread(s): 0.074 s  speedup 3.02
8 thread(s): 0.073 s  speedup 3.05
```

Each thread adds the square roots of its share of 400 million numbers.

Reading the results



- 2 threads: 1.98, almost perfect.
- 4 threads: 3.02: heat and serial work.
- 8 threads: 3.05: only 4 real cores.

Amdahl's law

If a part p of the program is parallel, on n cores:

$$S(n) = \frac{1}{(1-p) + \frac{p}{n}}$$

$p = 0.9$

$S(4) = \frac{1}{0.1+0.225} \approx 3.08$ (we measured 3.02!) $S(\infty) = \frac{1}{0.1} = 10.$

Benefits and challenges

Benefits	Challenges
more speed without a higher frequency	programs must use several threads
less energy for the same work	the serial part limits the speed-up
many programs run smoothly together	**synchronization** and **cache coherence**

Synchronization

Two threads read x = 5 and both write 6: one increase is lost. We need locks or atomic instructions.

Cache memory

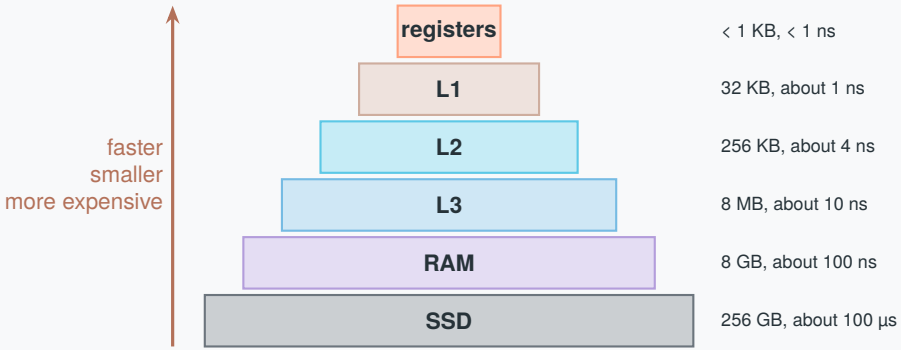
Definition

A small, fast memory near the processor that keeps a **copy** of recent data.

Analogy

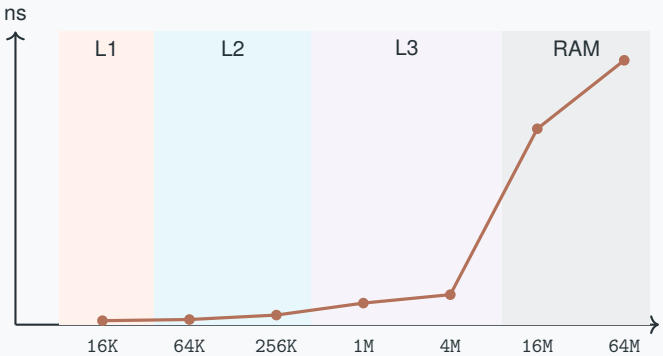
Desk (L1), bag (L2), cupboard (L3), library (RAM).

The memory hierarchy



A test: we can see the memory levels

```
   16 KB :   1.7 ns
   64 KB :   2.2 ns
  256 KB :   4.1 ns
 1024 KB :   9.2 ns
 4096 KB :  12.8 ns
16384 KB :  83.5 ns
65536 KB : 112.7 ns
```



From 1.7 ns (L1) to 113 ns (RAM): **65 times** slower.

Locality: why caches work

- **Temporal locality**: data used now will be used again soon.

- **Spatial locality**: after one address, we read the nearby ones.

Cache line

The cache moves blocks of **64 bytes**: reading `t[0]` also brings `t[1]` to `t[15]`.

A test: row by row or column by column?

```
23      t = now();
24      for (int i = 0; i < N; i++)       /* row by row: neighbours */
25          for (int j = 0; j < N; j++)
26              s += m[i][j];
27      printf("rows    : %.3f s\n", now() - t);
28
29      t = now();
30      for (int j = 0; j < N; j++)       /* column by column: jumps */
31          for (int i = 0; i < N; i++)
32              s += m[i][j];
```

```
rows    : 0.009 s
columns : 0.301 s
sum = 33554432
```

- C stores **row after row**.
- Columns: a new cache line every read.
- **33 times slower**!

Hits, misses, average time

Average access time

$$h \times t_{cache} + (1 - h) \times t_{RAM} \qquad (h = \text{hit rate})$$

$t_{cache} = 1$ **ns,** $t_{RAM} = 100$ **ns**

$h = 90\%$: 10.9 ns. $h = 99\%$: 1.99 ns, more than 5 times faster.

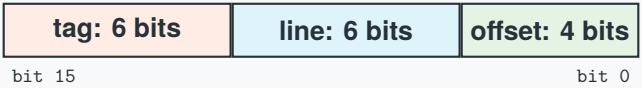
Mapping: where does a block go?

Kind	Rule	Note
direct mapped	one possible line	simple, but conflicts
fully associative	any line	few conflicts, costly search
set associative	any of k lines in a set	used in modern processors

Replacement: **LRU** (least recently used). Writing: **write-back** (fast) or write-through (simple).

Splitting an address (direct mapped)

1 KB cache, 16-byte lines, 16-bit addresses: 64 lines (6 bits), 16 bytes (4 bits), tag = 6 bits.



$1A2C_H = 000110\,100010\,1100_2$

tag = 6, line = 34, offset = 12.

Practice

Try it

1. $p = 0.75$: maximum speed-up? — 4
2. 2 ns cache, 80 ns RAM, 95% hits: average? — 5.9 ns
3. 4 KB cache, 32-byte lines, 32-bit addresses: tag bits? — 20

Summary

Remember

- More cores, not more GHz; Amdahl limits the gain.
- Memory: small and fast near, big and slow far.
- Read data in order: locality is speed.